UNIVERSITI PENDIDIKAN SULTAN IDRIS

اونيۏرسيتي ڤنديديقن سلطان ادريس

SULTAN IDRIS EDUCATION UNIVERSITY

Software Verification and Validation (DES3103)

by

Dr. Nor Asiah Mohamad@Razak

Course Learning Outline

1. Briefing Session (Ice Breaking via Whiteboard, Class rules & Pro forma): **M. Teams**

2. Introduction to Software Verification & Validation I & II (V&V)

3. Managing V&V with SQA Integration

4. Quality metrics or attributes in V&V

WEEK 6-MID-TERM TEST (QUIZ): Topic 1 – 3-(CLO1:C2) – 20 MARKS

WEEK 7-PHASE 1: ASSIGNMENT 1-ONLINE PROJECT-BASED LEARNING (OPBL)-(CLO3:A5) - 20 MARKS: M. Teams

WEEK 9-PHASE 1: PRESENTATION & REFLECTION ON PHASE 1: M. Teams

5. Software Reviews – Will be covered on Assignments 1 & 2

6. Quality Control: Software Testing - Will be covered in Assignments 1 & 2

7. Human-Computer Interface Testing - Will be covered in Assignments 1 & 2

8. Manual Testing and Automation Testing: Will be covered in Final Assessment

WEEK 11-PHASE 2: ASSIGNMENT 2-ONLINE PROJECT-BASED LEARNING (OPBL)-(CLO2&4:P4&C5) - 20 MARKS: M. Teams
WEEK 13-PHASE 2: PRESENTATION & REFLECTION ON PHASE 2: M. Teams

9. Quality Maturity Models

10. Software Problem Analysis and Reporting

11. Quality Culture in doing V&V.

12. Safety and Security in V&V

WEEK 14: REFLECTION ON IMPLEMENTATION OPBL (SOFTWARE TESTERS) IN THE V&V COURSE: e-FORUM

WEEK 16-PHASE 3: FINAL ASSESSMENT – INDIVIDUAL SPECIAL PROJECT BASED ON PHASE 2 (CLO4:C5) (2 WEEKS) - 40 MARKS : e-ASSESSMENT

UNIVERSITI PENDIDIKAN SULTAN IDRIS
اونيۏرسيتي ڤنديديقن سلطان ادريس
SULTAN IDRIS EDUCATION UNIVERSITY

TOPIC 8

Manual Testing vs. Automation Testing

- Overview of Manual vs Automation Testing
- Advantages & Disadvantages of Manual vs Automation Testing
- Software Testing Tools
- Software & System Testing Strategies
- Test Case Plan

TOPIC 8

UNIVERSITI PENDIDIKAN SULTAN IDRIS
SULTAN IDRIS EDUCATION UNIVERSITY

TABLE OF CONTENTS

Chapter 7: Test Automation 161

Benefits of Test Automation 161

Record and Playback Tools 162

Scripting Tools 163

Hybrid Tools 164

Frameworks 165

Automated Testing Tools 167

Automated Test Scripts 169

Summary 171

Check out the book here:

https://drive.google.com/file/d/1alz65MrAPHShfi7xmk5Ch_3ZBR76lGmC/view?usp=sharing

Chapter 2: Software Testing Types and Techniques 5

Functional Testing 5

Unit Testing 7

Integration Testing 9

System Testing 11

User Acceptance Testing 11

Nonfunctional Testing 13

Performance Testing 14

Security Testing 17

Usability Testing 20

Compatibility Testing 21

Scalability Testing 22

Reliability Testing 23

Availability Testing 24

Installability Testing 24

Maintainability Testing 26

Compliance Testing 27

Static Testing 29

Code Reviews 29

Requirement Reviews 30

Design Reviews 31

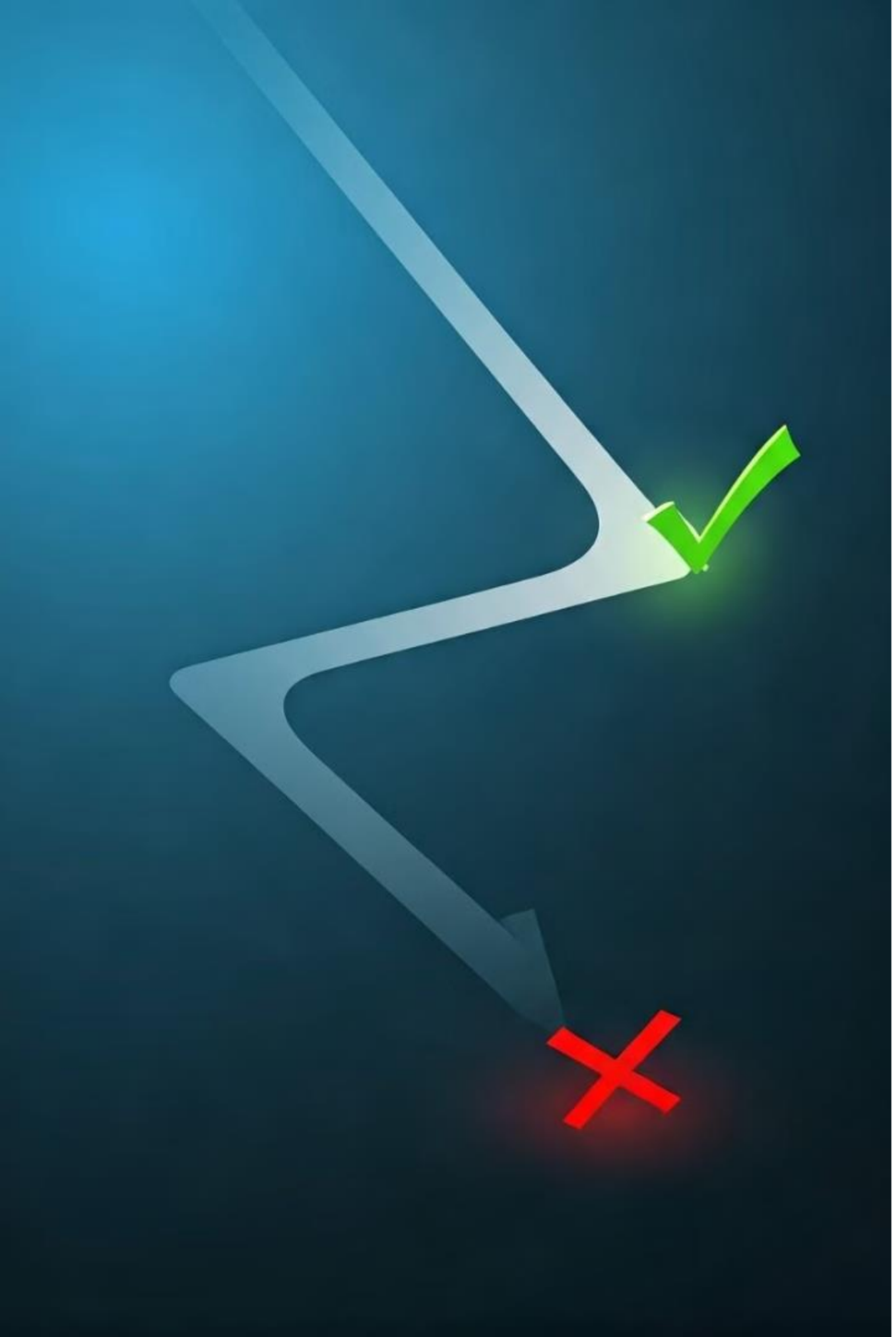
Walk-Throughs 32

Inspections 33

Summary 34

Manual Testing vs Automation Testing

Manual testing and automation testing are **two distinct approaches** used in **software testing** to ensure the quality of a software application. While both methods have their own advantages and disadvantages, they differ significantly in their approach, execution, and benefits.



When to Use Each

Manual testing is best used for exploratory testing, usability testing, or when the application is still in the early stages of development. Automation testing is best used for regression testing, smoke testing, or when the application has a large number of test cases that need to be executed repeatedly.

1

Manual Testing

Exploratory testing, usability testing, early stages of development.

2

Automation Testing

Regression testing, smoke testing, large number of test cases.

Manual Testing

Manual testing involves testing a software application manually, without using any automated **tools or scripts**. In this approach, a **human tester** executes a set of **pre-defined** test cases, observes the results, and reports any defects or errors found.

Manual Testing Tools

While manual testing doesn't require any specific tools, there are some **tools that can aid in the process**. These tools can **help teams organize, run, and track manual tests**, as well as **track and manage defects**.

Test Management Tools

TestRail, PractiTest, TestLink

Defect Tracking Tools

JIRA, Trello, Bugzilla

Screen Recording and Capture Tools

Camtasia, OBS Studio, Snagit

Test Management Tools

1 TestRail

Organizes, runs, and tracks manual tests.

www.testrail.com

2 PractiTest

Provides a platform for manual testing, test case management, and and defect tracking.

www.practitest.com

3 TestLink

An open-source test management tool for managing and executing manual tests.

www.testlink.org

Defect Tracking Tools

1 JIRA

A popular project management tool with defect tracking capabilities.

www.atlassian.com

2 Trello

A visual project management tool for defect tracking and management.

www.trello.com

3 Bugzilla

A defect tracking tool for tracking and managing defects.

www.bugzilla.org

Screen Recording and Capture Tools

Camtasia

A screen recording and video editing tool for recording manual testing sessions.

www.techsmith.com

OBS Studio

A free and open-source screen recording tool for recording manual testing sessions.
sessions.

www.obsproject.com

Snagit

A screen capture and recording tool for documenting testing sessions.

www.techsmith.com/snagit

Advantages & Disadvantages of Manual Testing

Advantages

Exploratory Testing: Manual testing allows for exploratory testing, where testers can discover issues that might not be caught through automation.

Human Intuition: Human testers can use their intuition and experience to identify issues that might not be easily replicable through automation.

Flexibility: Manual testing can be adapted quickly to changing requirements or new features.

Disadvantages

Time-consuming: Manual testing can be a time-consuming and labor-intensive process.

Error-prone: Human testers can make mistakes, which can lead to errors or missed defects.

Limited coverage: Manual testing can only cover a limited number of test cases and scenarios.

Automation Testing

Automation testing involves using **software tools** or **scripts** to execute **pre-scripted tests** on a software application. These tools can simulate user interactions, verify expected results, and report any defects or errors found.

Automation Testing Tools

Many automation testing tools are available, each with strengths and weaknesses. These tools can be used for unit testing, functional testing, API testing, and test automation frameworks.

Unit Testing Frameworks	JUnit (Java), PyUnit (Python), NUnit (.NET)
Functional Testing Tools (Web and Mobile Automation Tools)	Selenium, Appium, Cypress
API Testing Tools	Postman, SoapUI, Rest Assured
Test Automation Frameworks	TestNG, Cucumber, Robot Framework

Unit Testing Frameworks

JUnit (Java)

A popular unit testing framework for Java.

www.junit.org

PyUnit (Python)

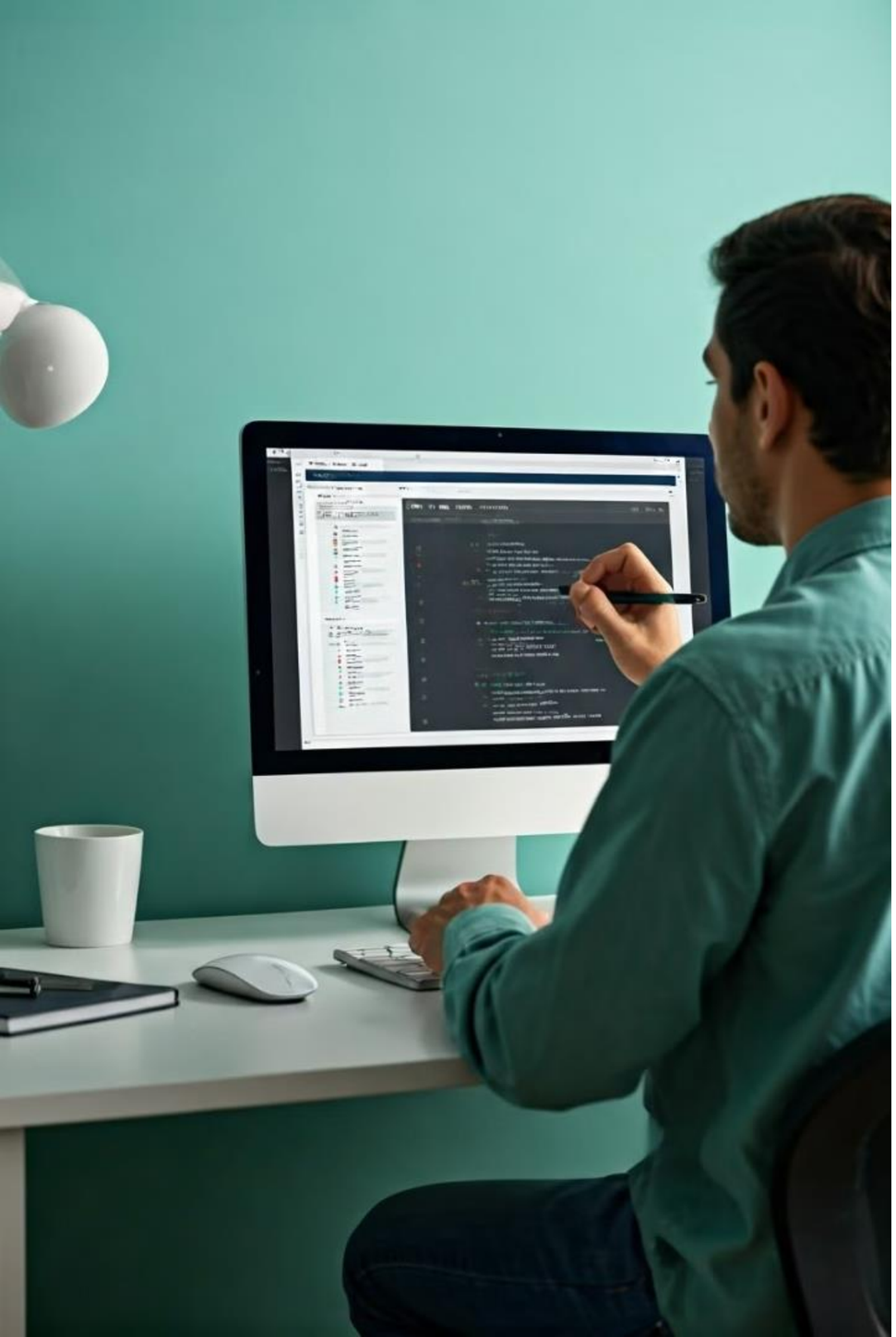
A unit testing framework for Python. Python.

www.python.org

NUnit (.NET)

A unit testing framework for .NET.

www.nunit.org



Web and Mobile Automation Tools Tools

1

Selenium

An open-source tool for automating web browsers, allowing testers to simulate user interactions and perform functional tests, facilitating the testing of web applications accessed through desktop browsers

www.selenium.dev

2

Appium

An open-source tool for automating native, mobile, and web applications. A applications. A cross-platform automation framework for testing native, native, hybrid, and web applications on iOS and Android devices.

www.appium.io

3

Cypress

A fast and easy-to-use test automation framework for web applications. applications.

A modern JavaScript testing framework specifically designed for automating automating web applications, offering features like automatic waiting and waiting and real-time feedback.

www.cypress.io

API Testing Tools

Postman	A popular API testing testing tool for API testing and documentation.	www.postman.com
SoapUI	A comprehensive API API testing tool for API API testing, functional functional testing, and and load testing.	www.soapui.org
Rest Assured	A Java-based API testing tool for API validation.	www.rest-assured.io

Test Automation Frameworks

1

TestNG

A Java-based test automation framework for unit testing, functional testing, and integration testing via desktop applications..

www.testng.org

2

Cucumber

A behavior-driven development (BDD) framework that allows testers to write automated tests using a natural language syntax, making tests more readable and maintainable.
www.cucumber.io

3

Robot Framework

A generic test automation framework suitable for testing web, mobile, and desktop applications
www.robotframework.org

Advantages & Disadvantages of Automation Testing

Advantages

Faster execution: Automation testing can execute tests much faster than manual testing.

Increased coverage: Automation testing can cover a large number of test cases and scenarios, including those that are difficult or impossible to test manually.

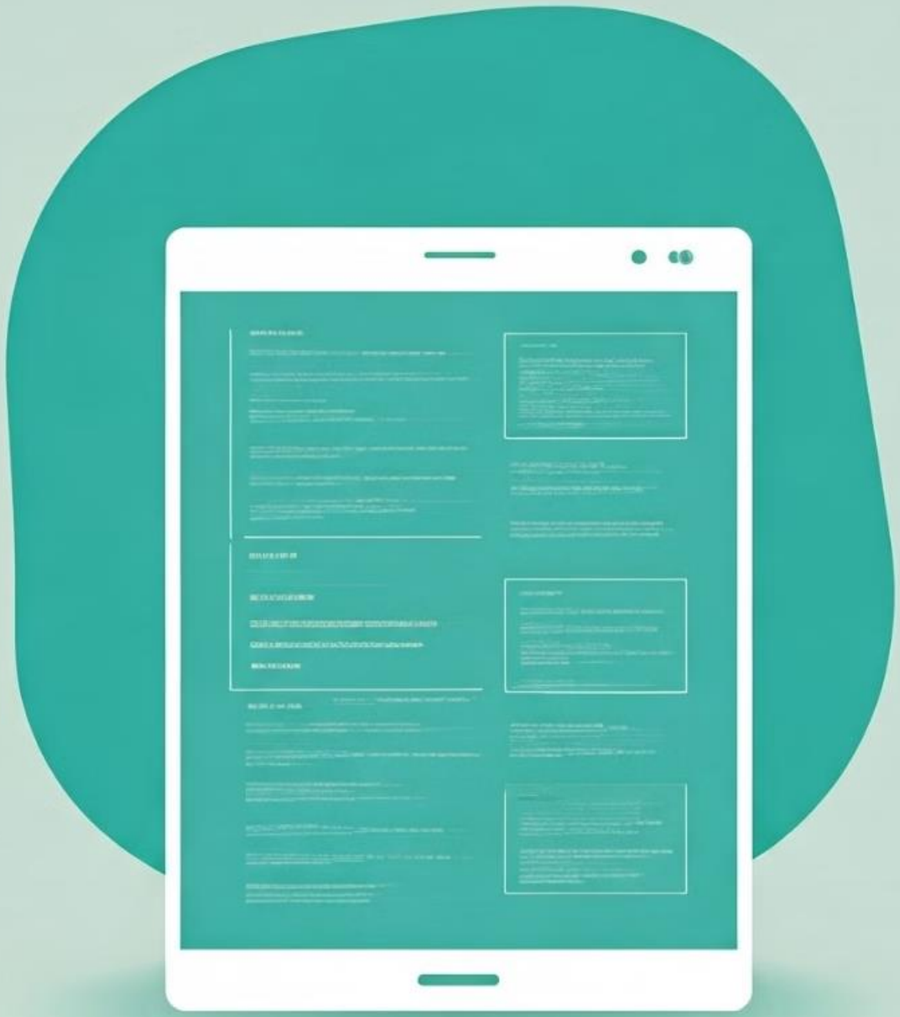
Repeatability: Automation testing can repeat tests multiple times, ensuring consistency and reliability.

Disadvantages

Initial investment: Automation testing requires an initial investment in tools, training, and script development.

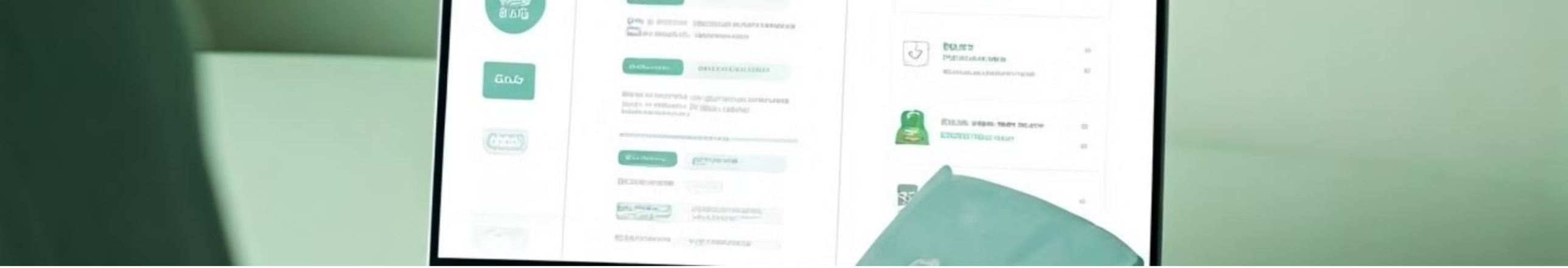
Maintenance: Automation tests require maintenance to ensure they continue to work as the application changes.

Limited scope: Automation testing may not be able to cover all types of testing, such as exploratory testing or usability testing.



Software and System Testing Strategies

Software testing is crucial for ensuring the **quality** and **reliability** of
of software systems. This presentation explores into the various types of
types of software and systems that can be effectively tested using both
both manual and automated testing techniques. This includes **web**
web applications, **mobile apps**, **desktop applications**, and **APIs**.



Web Applications Testing

1 E-commerce Websites (e.g., Amazon, eBay)

Functional testing verifies shopping cart functionality, payment processing, and order tracking. **Performance testing** assesses website speed and responsiveness under high traffic loads.

2 Social Media Platforms (e.g., Facebook, Twitter)

Security testing focuses on user account protection, data privacy, and privacy, and preventing malicious attacks. **Usability testing** evaluates the evaluates the ease of use, navigation, and user experience.

3 Online Banking Systems (e.g., PayPal, Bank of America)

Security testing is paramount for protecting sensitive financial information, ensuring secure transactions, and preventing fraud. fraud.

4 Online Learning Platforms (e.g., Coursera, Udemy)

Functionality testing verifies course enrollment, content delivery, and delivery, and student progress tracking. **Usability testing** assesses the assesses the user interface and learning experience.

Mobile Applications Testing

Native Mobile Apps

(e.g., Instagram, Facebook)

Performance testing ensures smooth smooth app performance on various various mobile devices and operating operating systems. **Compatibility testing** verifies that the app functions functions correctly across different devices and screen sizes.

Hybrid Mobile Apps

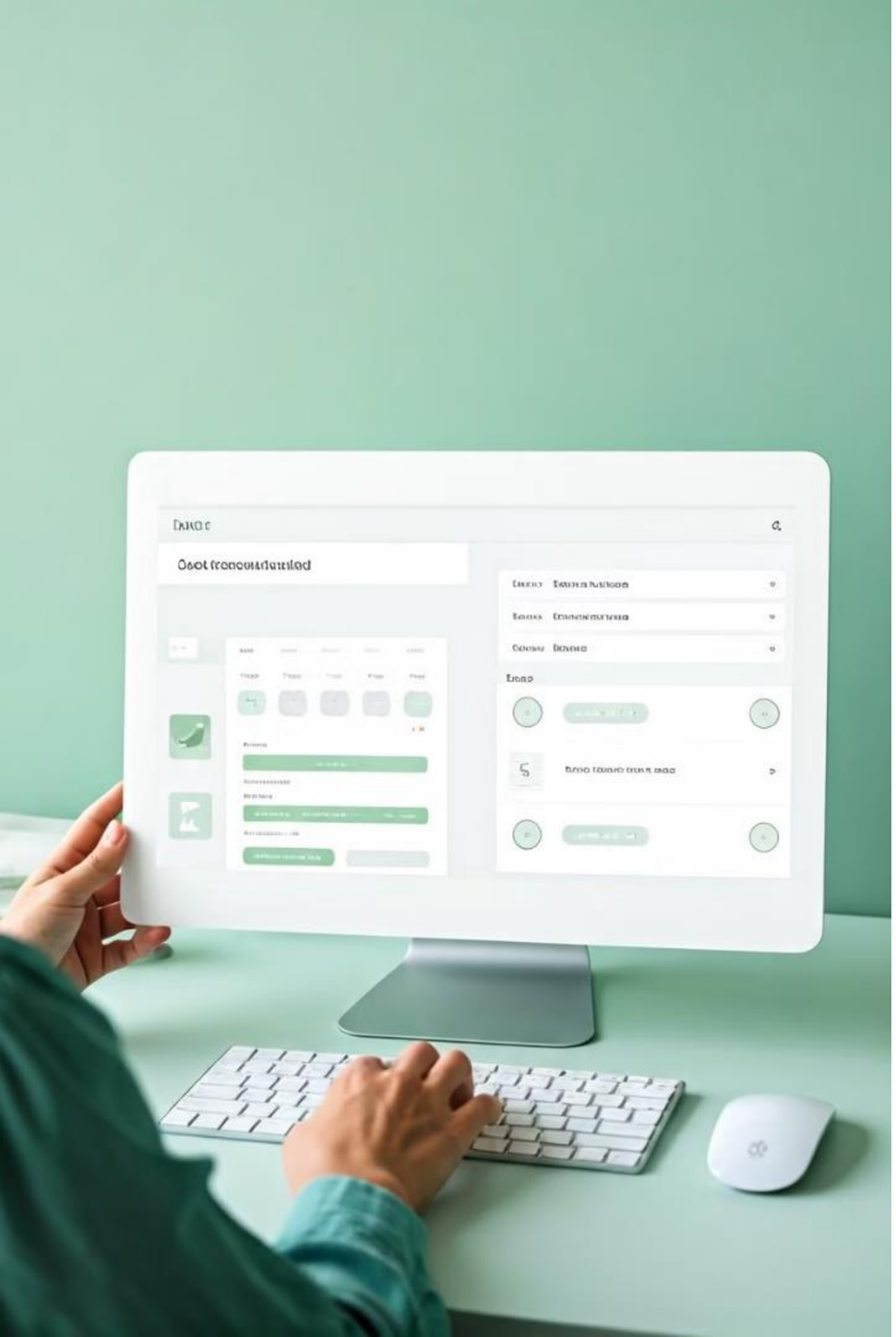
(e.g., Twitter, LinkedIn)

Functionality testing verifies that the app's features work as expected on both web and mobile platforms. **Cross-platform compatibility testing** ensures seamless operation across multiple devices and operating systems.

Mobile Games

(e.g., Pokémon Go, Candy Crush)

Performance testing ensures smooth smooth gameplay, loading times, and and responsiveness. **Usability testing testing** evaluates the game's controls, controls, interface, and overall user user experience.



Desktop Applications Testing

Microsoft Office (e.g., Word, Excel, PowerPoint)

Functionality testing verifies features like document creation, editing, formatting, and and collaboration. **Usability testing** assesses the ease of use, intuitive interface, and workflow and workflow efficiency.

Adobe Creative Cloud (e.g., Photoshop, Illustrator, InDesign)

Performance testing ensures smooth operation and responsiveness for demanding tasks like demanding tasks like image editing and video rendering. **Compatibility testing** verifies that the verifies that the software works with different file formats and hardware configurations. configurations.

Antivirus Software (e.g., Norton Antivirus, McAfee Antivirus)

Security testing evaluates the software's ability to detect and remove malware. **Performance** **Performance testing** assesses the software's impact on system performance and resource resource consumption.

API Testing

API Type	Testing Focus
RESTful APIs (e.g., Twitter API, Facebook API)	Functional, performance, security, and integration testing testing
SOAP APIs (e.g., Amazon Web Services API, Google Maps API)	Functional, security, and interoperability testing
GraphQL APIs (e.g., GitHub API, Shopify API)	Functional, performance, security, and schema validation validation testing

Test Case Plan

A test case plan is a document that outlines the test cases that will be used be used to test a software application. It includes the **test case ID, description, type, priority, estimated time, test data, expected result, result, actual result, and status.**

Made with Gamma

Test Case Template

Test Case ID

A unique identifier for the test case.

Test Case Description

A concise description of the purpose of purpose of the test case.

Test Objective

The specific goal or objective of the test.

Made with Gamma

Test Case Purposes

1 Ensure Requirements

The software meets the specified requirements and functions as intended.

2 Establish Standards

A standard for the software's behavior and performance, ensuring that it meets a certain level of quality before being released to users.

3 Detailed Record

Provide a detailed record of how the software was tested.

Test Case Components

Test Case ID	Title/Description	Test Objective
Preconditions	Test Steps	Input Data
Expected Results	Actual Results	Status

Manual Testing vs Automation Testing Test Case Plan

Manual Testing

Identify Objective: Identify the test objective and preconditions.

Determine Steps: Determine the test steps and input data.

Define Results: Define the expected results.

Execute Steps: Execute the test steps and record the actual results.

Compare Results: Compare the actual results with the expected results.

Automation Testing

Identify Objective: Identify the test objective and preconditions.

Determine Steps: Determine the test steps and input data.

Define Results: Define the expected results.

Write Script: Write the automation script using a programming language and testing framework.

Execute Script: Execute the automation script and record the actual results.

Compare Results: Compare the actual results with the expected results.

Manual Testing

Identify Objective: Identify the test objective and preconditions.

Determine Steps: Determine the test steps and input data.

Define Results: Define the expected results.

Execute Steps: Execute the test steps and record the actual results.

Compare Results: Compare the actual results with the expected results.

Automation Testing

Identify Objective: Identify the test objective and preconditions.

Determine Steps: Determine the test steps and input data.

Define Results: Define the expected results.

Write Script: Write the automation script using a programming language and testing framework.

Execute Script: Execute the automation script and record the actual results.

Compare Results: Compare the actual results with the expected results.

Test Plan Sample for Automation Software Testing

No.	Section	Description
1.	Test Case ID	TP-001
2.	Test Case Name	**Login Functionality**
3.	Test Case Description	Verify that the login functionality of the web application works correctly.
4.	Preconditions	The web application is installed and running on the test environment. The test data is available and correct.
5.	Test Steps	Launch the web application in the browser. Enter a valid username and password in the login form. Click the login button. Verify that the user is logged in successfully and redirected to the dashboard page.
6.	Expected Results	The user is logged in successfully and redirected to the dashboard page. The login form is validated correctly, and error messages are displayed for invalid inputs.
7.	Test Data	Valid username: testuser. Valid password: testpassword. Invalid username: invaliduser. Invalid password: invalidpassword.

No.	Section	Description
8.	Automation Tool	Selenium WebDriver
9.	Programming Language	Java
10.	Test Framework	TestNG
11.	Test Script	Java

```java
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.Test;

public class LoginTest {
    WebDriver driver = new ChromeDriver();

    @Test
    public void loginTest() {
        driver.get("https://example.com/login");
        WebElement usernameInput = driver.findElement(By.id("username"));
        WebElement passwordInput = driver.findElement(By.id("password"));
        WebElement loginButton = driver.findElement(By.id("login-button"

        usernameInput.sendKeys("testuser");
        passwordInput.sendKeys("testpassword");
        loginButton.click();

        String dashboardTitle = driver.getTitle();
        Assert.assertEquals(dashboardTitle, "Dashboard");
    }
}
```

No.	Section	Description
12.	Test Environment	Operating System: Windows 10. Browser: Google Chrome. Web Application: Example Web Application.
13.	Test Schedule	Test execution: 1 hour. Test reporting: 30 minutes.
14.	Test Deliverables	Test report in HTML format. Test script in Java format.

Test Plan Sample for Automation Software Testing

No.	Section	Description
1.	Test Case ID	TP-002
2.	Test Case Name	**Payment Processing**
3.	Test Case Description	Verify that the payment processing functionality of the web application works correctly.
4.	Preconditions	• The web application is installed and running on the test environment. • The test data is available and correct. • The payment gateway is configured correctly.
5.	Test Steps	1. Launch the web application in the browser. 2. Add a product to the shopping cart. 3. Proceed to the checkout page. 4. Enter valid payment information (e.g., credit card number, expiration date, CVV). 5. Click the "Pay" button. 6. Verify that the payment is processed successfully and the order is confirmed. 7. Verify that the payment information is stored correctly in the database.
6.	Expected Results	• The payment is processed successfully and the order is confirmed. • The payment information is stored correctly in the database. • Error messages are displayed for invalid payment information.
7.	Test Data	• Valid credit card number: 4111 1111 1111 1111 • Valid expiration date: 12/2025 • Valid CVV: 123 • Invalid credit card number: 4111 1111 1111 1112 • Invalid expiration date: 12/2020 • Invalid CVV: 124

No.	Section	Description
8.	Automation Tool	Selenium WebDriver
9.	Programming Language	Java
10.	Test Framework	TestNG

11.	Test Script	Java
		java Verify Open in Editor
		1 import org.openqa.selenium.By;
		2 import org.openqa.selenium.WebDriver;
		3 import org.openqa.selenium.WebElement;
		4 import org.openqa.selenium.chrome.ChromeDriver;
		5 import org.testng.annotations.Test;
		6
		7 public class PaymentTest {
		8 WebDriver driver = new ChromeDriver();
		9
		10 @Test
		11 public void paymentTest() {
		12 driver.get("https://example.com/checkout");
		13 WebElement productNameInput = driver.findElement(By.id("product
		14 WebElement paymentInfoInput = driver.findElement(By.id("payment
		15 WebElement payButton = driver.findElement(By.id("pay-button"));
		16
		17 productNameInput.sendKeys("Test Product");
		18 paymentInfoInput.sendKeys("4111 1111 1111 1111");
		19 payButton.click();
		20
		21 String orderConfirmationMessage = driver.findElement(By.id("ord
		22 Assert.assertEquals(orderConfirmationMessage, "Order confirmed
		23
		24 // Verify payment information is stored correctly in the databa
		25 // ...
		26 }
		27 }

12.	Test Environment	• Operating System: Windows 10 • Browser: Google Chrome • Web Application: Example Web Application • Payment Gateway: PayPal
13.	Test Schedule	• Test execution: 2 hours • Test reporting: 1 hour
14.	Test Deliverables	• Test report in HTML format • Test script in Java format

Functionality for Automation Testing

User Interface (UI) Testing

Clicking buttons and links, filling out forms and submitting data, verifying UI elements.

API Testing

Sending API requests and verifying responses, testing API endpoints, methods, and parameters.

Database Testing

Verifying data insertion, update, and deletion, testing database queries and stored procedures.

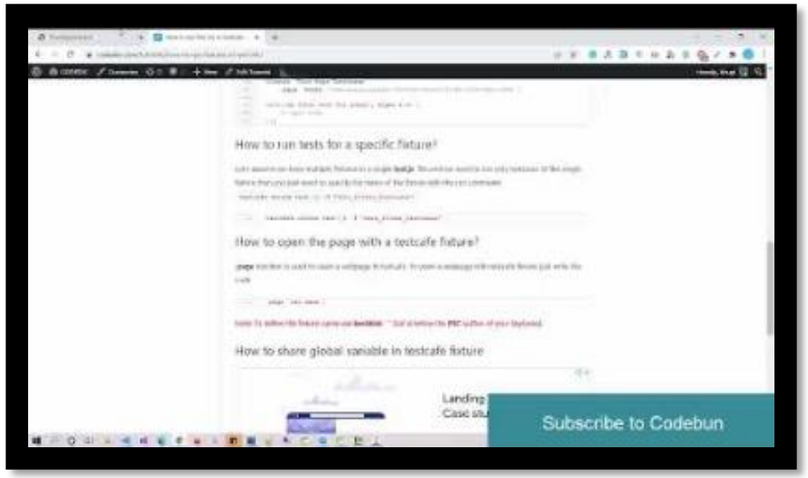
Mobile Application Testing

Testing mobile app UI and functionality, verifying mobile app performance and battery life.

Made with Gamma

Example of automated testing tool- TestCafe

Login & registration form



https://youtu.be/Xy7ph-4JTA0

API

https://youtu.be/fXitpuPpzNY

UI & API

https://youtu.be/6wQRSSs6biQ

Example of automated testing tool- pCloudy

- https://youtu.be/mev1JgAWh7I

References

- Top 16 Automation Testing Tools 2024 | Katalon
- Automation Testing Tools: Definition, Benefits, & Examples (kms-solutions.asia)
- What Is Automated Testing and How Does It Work? (With Example) (testdevlab.com)
- Automated Testing - Which Tests to Automate [+ Examples] (globalapptesting.com)
- 10 Best Automation Testing Tools of 2024 (functionize.com)
- Manual & Automated Testing: Test Automation Use Cases (testlio.com)
- Best Automation Testing Tools for 2024 | BrowserStack
- Automation Testing - Software Testing – GeeksforGeeks
- Best 7 Automated Software Testing Tools in 2024 (mailtrap.io)
- Top 10 Manual Testing Tools: Boost Your QA Efficiency (kiwiqa.com)

END

UNIVERSITI PENDIDIKAN SULTAN IDRIS
اونيۏرسيتي ڤنديديقن سلطان ادريس
SULTAN IDRIS EDUCATION UNIVERSITY

Thank You…
Terima kasih..